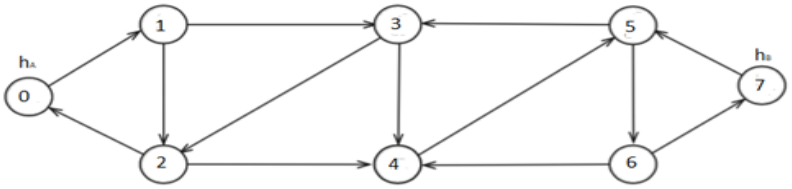
b. Two spy, Ahmad and and Baber have been stationed in the enemy territory at houses h_A and h_B respectively. Ahmed needs to hand over a secret information to Baber. It is decided that Ahmed and Baber will meet each other at a third location, a hotel, where the information will be exchanged. The secret agency has already prepared a road map of the enemy territory. It contains the homes h_A, h_B, and the n hotels in the area: h1, h2, …, hn. This map formulates a graph where each location (houses and hotels) are its vertices and if one location is directly reachable via a road from the other then there is an edge between them. Each edge (road) poses a risk of being caught. The total risk of a path is simply the number of edges on that path.

The agency wishes to tell Ahmed and Baber which hotel, h, to meet at so that the total risk, for both of them combined, is minimized. Such a hotel would be the *safest hotel*. Note that both Ahmed and Baber would need to travel to h to make the delivery possible. The problem is that the map is too large for manual processing. Therefore, you have been tasked **to write a C++ program** that can find and return the id of the safest hotel given the graph G and locations h_A and h_B. ***Your goal is to write a function in Graph class that takes no more than O((\|V\|+\|E\|)) time to accomplish this task. If you use any helper function then also provide its code.*** You can assume that graph is represented as adjacency list and all the vertices are labelled 0 to |V|-1. Consider the example below: Vertex 0 is h_A and vertex 7 is h_B, here the safest hotel is vertex 3 and the minimum total risk is 4.




```
#include <list>
class Graph{
      int V;   // No. of vertices
      list<int> *adjList; //adjacency lists of neighbours (std list)
public:
      Graph(int V);  // Constructor
     … SafestHotel(…) // think of input parameter and return type
};
```

```
int safestHotel(int ha, int hb) {
            int* d1 = new int[V];
            int* d2 = new int[V];

            BFS(ha, d1);
            BFS(hb, d2);
            int min = 0;
            for (int i = 1;i < V;i++) {
                  if (d1[min] + d2[min] > d1[i] + d2[i])
                        min = i;
            }
            return min;
      }
```

```
void BFS(int s, int* d) {
            bool* visited = new bool[V];
            for (int i = 0;i < V; i++)
                  visited[i] = false;
            d[s] = 0;
            queue<int> q;
            q.push(s);
            visited[s] = true;
            while (!q.empty()) {
                  int v = q.front();
                  q.pop();
                  list<int>::iterator it;
                  for (it = adjList[v].begin(); it != adjList[v].end();it++) {
                        int u = *it;
                        if (visited[u] == false) {
                              visited[u] = true;
                              d[u] = d[v] + 1;
                              q.push(u);
                        }
                  }
            }
      }
```